

Data Transfer

COMPLECS Team

<https://bit.ly/COMPLECS>

<https://github.com/sdsc-complecs>

COMPLECS NSF award #2320934

SDSC
SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego

About COMPLECS

COMPLECS (COMPrehensive Learning for end-users to Effectively utilize CyberinfraStructure) is a new training program offered by SDSC that covers the most important ***non-programming*** concepts and skills that you need to effectively use supercomputers. Topics include parallel computing concepts, Linux tools and bash scripting, security, batch and interactive computing, how to get help, and data management.

COMPLECS is supported by NSF [CISE/OAC-2320934](#)



COMPLECS: Data Management Series

Data Storage and File Systems

How to use the data storage and file systems you'll find mounted on high-performance computing systems.

Data Transfer

How to get the data you need for your research to and from high-performance computing systems.

Parallel I/O

How to make effective use of parallel file systems.



Data Management is a HUGE topic



Data management is a field of study in its own right. As part of the COMPLECS training program, we will only be able to address a limited set of the most essential data management topics as they relate to high-performance computing. If your research involves complex data management issues, then we highly encourage you to seek out the advice of experts at your own institution and/or refer to additional resources that may be available online.

[Image Credit: Harvard Biomedical Data Management](#)



Data Transfer

Learning Goals:

- Download data from the Internet
- Check the integrity of your data
- How to use data compression
- Key data transfer tools
- Data distribution networks and federations

Prerequisite Knowledge:

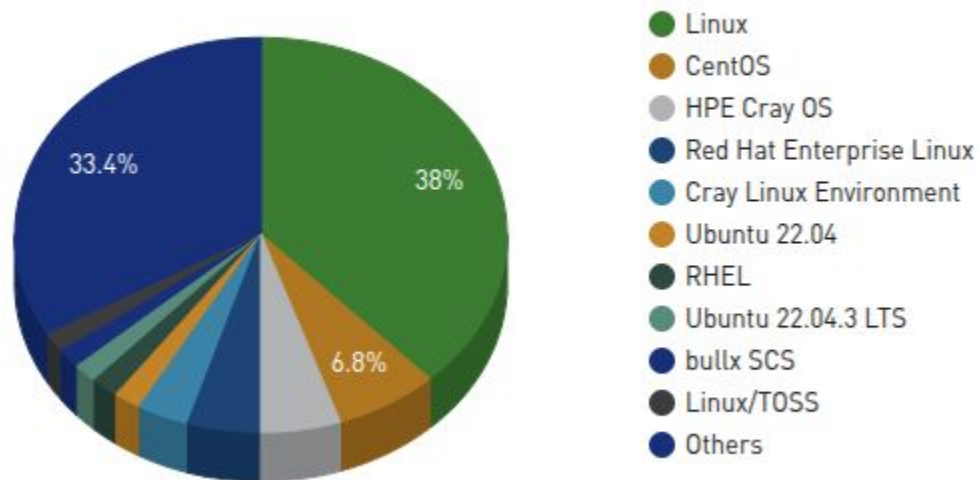
- [Intermediate Linux and Shell Scripting](#)
- [Data Storage and Filesystems](#)

HPC runs on Linux

High-performance computing (HPC) and advanced cyberinfrastructure (CI) runs on [Linux](#).

If you don't believe us, then look no further than [the latest statistics from the TOP500](#) --- a list of the most powerful supercomputers in the world.

Operating System System Share



Throughout COMPLECS training program, we highlight the use of many standard command-line tools and applications that are available for [Unix-like](#) operating systems such as Linux and [macOS](#).

Recommendation for Windows users: Install the [Windows Subsystem for Linux](#) on your personal computer to gain more experience with Unix-based systems.

What is Data Transfer?

When we talk about ***data transfer*** in the context of scientific and high-performance computing, we typically use it as a synonym for ***file transfer*** – the process of transmitting a ***file*** from one computer system to another over a network communication channel, which is mediated by a ***file transfer protocol***.

A *data transfer* may also refer to an individual instance of a *file transfer*.

Key components and/or characteristics in any *data transfer*:

- **Source** - Where is the data located?
- **Format** - How is the data organized?
- **Size** - How much data is there?
- **Destination** - Where is the data going?
- **Method** - How should we transfer the data?

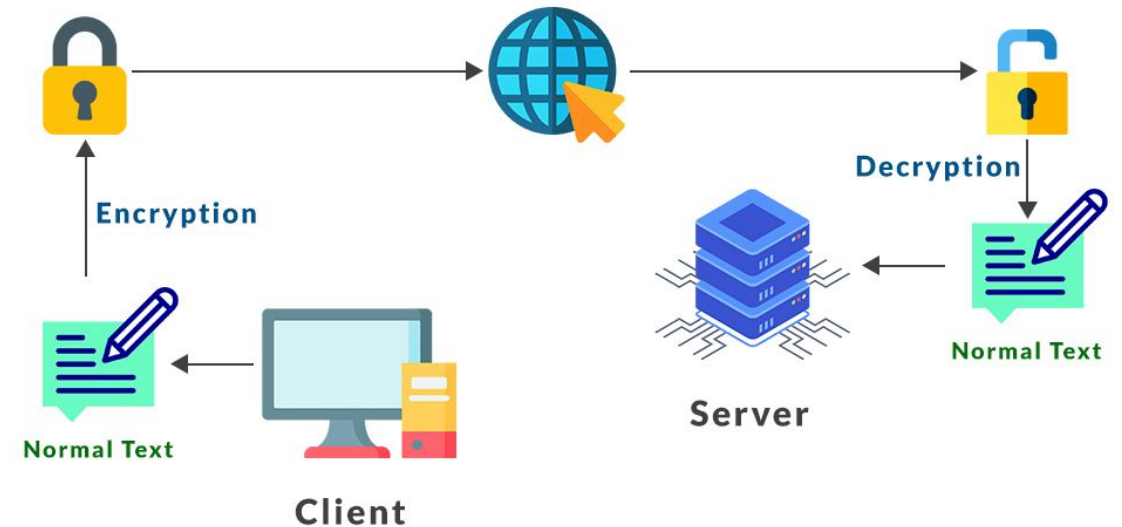


Table of Contents

- **Downloading Data from the Internet**
 - **wget - non-interactive network downloader**
 - **curl - transfer a URL**
 - **aria2c - ultra fast download utility**
- **Data Integrity and Checksums**
- **Data Compression and Archives**
- **Data Transfer Tools**
- **Data Distribution Networks and Federations**
- **Questions & Answers (30 min)**

wget - non-interactive network downloader

wget is a simple command-line tool for downloading files from the web. It supports [HTTP/S](#) and [FTP](#) protocols as well as file retrieval through [proxy servers](#). It has been designed for robustness over slow or unstable network connections. If a download fails due to a network problem, you can setup the download to keep retrying, resuming the download from where you left off until the whole file has been retrieved.

Download a file

```
$ wget https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
```

Download multiple files

```
$ wget -i cifar-100.txt
```

```
$ cat cifar-100.txt
https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz
https://www.cs.toronto.edu/~kriz/cifar-100-matlab.tar.gz
https://www.cs.toronto.edu/~kriz/cifar-100-binary.tar.gz
```

Resume a download

```
$ wget -c https://www.image-net.org/data/ILSVRC/2012/ILSVRC2012_img_train.tar
```

curl - transfer a URL

curl is a sophisticated command-line tool (and software library) for transferring data from or to a server [using various network protocols](#). By default, curl invokes an [HTTP/S GET request](#) on a URL, displaying the retrieved data to standard output.

Download a file

```
$ curl -O https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
```

Download multiple files

```
$ curl -K cifar-100.txt
```

```
$ cat cifar-100.txt
-O
url = "https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz"
-O
url = "https://www.cs.toronto.edu/~kriz/cifar-100-matlab.tar.gz"
...
```

Resume a download

```
$ curl -C - -O https://www.image-net.org/data/ILSVRC/2012/ILSVRC2012_img_train.tar
```

Download multiple files in parallel

```
$ curl -Z -O 'https://www.cs.toronto.edu/~kriz/cifar-100-{python,matlab,binary}.tar.gz'
```

aria2c - ultra fast download utility

[aria2c](#) is a command-line tool for downloading files. It supports both [HTTP/S](#) and [S/FTP](#) as well as [BitTorrent](#). More interestingly, it can download a file from multiple sources over different protocols in parallel and tries to utilize your maximum download bandwidth.

Download a file

```
$ aria2c https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
```

Download multiple files

```
$ aria2c -i cifar-100.txt
```

```
$ cat cifar-100.txt
https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz
https://www.cs.toronto.edu/~kriz/cifar-100-matlab.tar.gz
https://www.cs.toronto.edu/~kriz/cifar-100-binary.tar.gz
```

Resume a download

```
$ aria2c -c https://www.image-net.org/data/ILSVRC/2012/ILSVRC2012_img_train.tar
```

Download a file in parallel with multiple connections per host

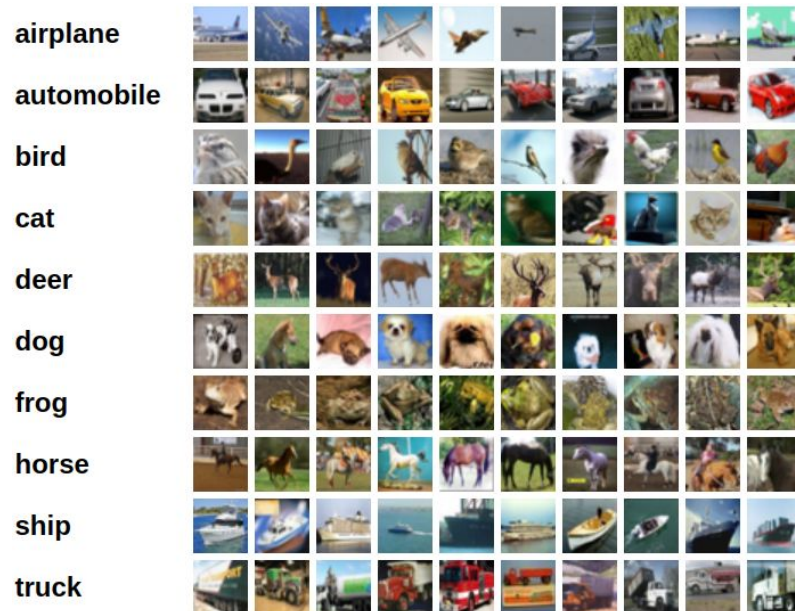
```
$ aria2c -x2 https://www.image-net.org/data/ILSVRC/2012/ILSVRC2012_img_train.tar
```

Table of Contents

- Downloading Data from the Internet
- **Data Integrity and Checksums**
 - **md5sum - compute and check MD5 message digest**
 - **sha256sum - compute and check SHA256 message digest**
- Data Compression and Archives
- Data Transfer Tools
- Data Distribution Networks and Federations
- Questions & Answers (30 min)

Data Integrity

Data integrity is related to the preservation of data consistency over time. It is the opposite of data corruption. For example, upon downloading a file from a given source, data integrity techniques may be used to ensure the copy of the file you have obtained from the source is the same as when it was originally created and/or recorded at the source.

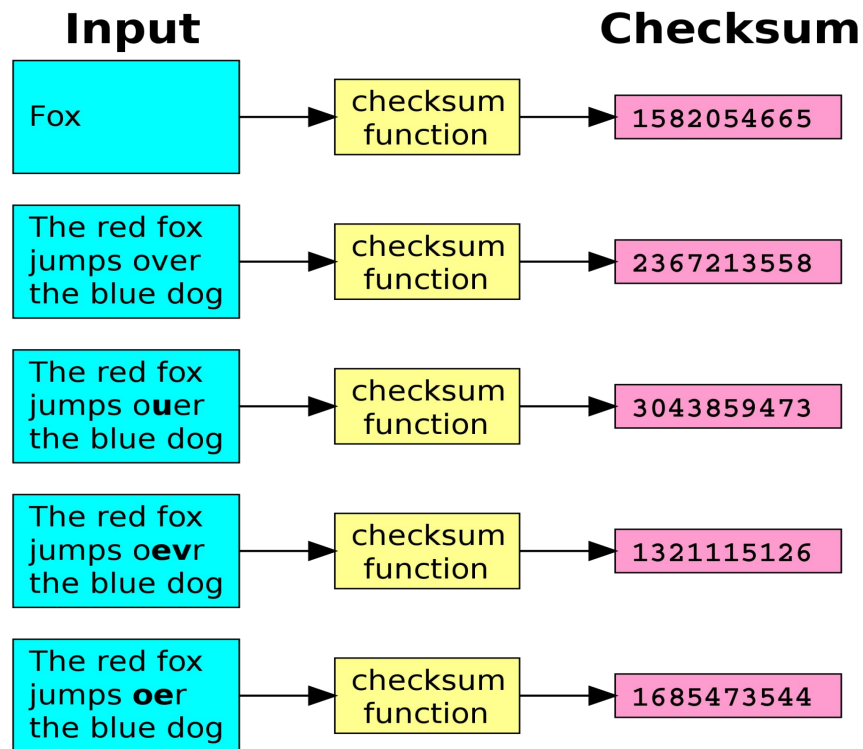


So how do you check the integrity of a file you've downloaded from the Internet and ensure it has not been corrupted?

<https://www.cs.toronto.edu/~kriz/cifar.html>

Checksum

A [checksum](#) is a small block of data — almost like to a unique fingerprint or ID – derived from another, larger block of data, which may then be used to detect any errors introduced into the larger data block either during data transfer and/or long-term storage of the data.



Checksums are used to verify data integrity.

Checksum algorithms (or functions) are typically implemented using [hash functions](#), which are functions that can map input data of arbitrary size to a fixed-sized output value known as a *hash value*, *digest*, or simply, *hashes*.

Checksums should be easy to compute and minimize duplicate output values for different inputs — a problem known as a [hash collision](#).

md5sum - compute & check MD5 message digest

[md5sum](#) is a command-line tool that calculates and verifies a 128-bit [MD5](#) hash value of a file. While you will commonly find MD5 hashes still in use, please note that the underlying MD5 algorithm is no longer deemed to be secure.

Version	Size	md5sum
CIFAR-10 python version	163 MB	c58f30108f718f92721af3b95e74349a
CIFAR-10 Matlab version	175 MB	70270af85842c9e89bb428ec9976c926
CIFAR-10 binary version (suitable for C programs)	162 MB	c32a1d4ab5d03f1284b67883e8d87530

Compute the MD5 hash of a file

```
$ md5sum cifar-10-python.tar.gz
```

Compute the MD5 hashes of multiple files and output them to a file

```
$ md5sum cifar-10-python.tar.gz cifar-10-matlab.tar.gz cifar-10-binary.tar.gz > cifar-10.md5
```

Check and verify an MD5 hash file

```
$ md5sum -c cifar-10.md5
```

* macOS has an equivalent command-line tool named md5.

sha256sum - compute & check SHA256 message digest

sha256sum is a command-line tool that calculates and verifies a 256-bit [SHA256](#) hash value of a file. It is a cryptographic hash function, designed to detect internal tampering, and is the recommended checksum utility you should use over md5sum.

Compute the SHA256 hash of a file

```
$ sha256sum cifar-100-python.tar.gz
```

Compute the SHA256 hashes of multiple files and output them to a file

```
$ sha256sum cifar-100-python.tar.gz \  
             cifar-100-matlab.tar.gz \  
             cifar-100-binary.tar.gz > cifar-100.sha256
```

Check and verify a SHA256 hash file

```
$ sha256sum -c cifar-100.sha256
```

```
$ cat cifar-100.sha256  
85cd44d02ba6437773c5bbd22e183051d648de2e7d6b014e1ef29b855ba677a7  cifar-100-python.tar.gz  
5fb8d16a77f63e3e417308497f2dcd58fca65b85b8c765ee02ec4e01c9f1d0d6  cifar-100-matlab.tar.gz  
58a81ae192c23a4be8b1804d68e518ed807d710a4eb253b1f2a199162a40d8ec  cifar-100-binary.tar.gz
```

* macOS has an equivalent command-line tool named sha256.

Table of Contents

- Downloading Data from the Internet
- Data Integrity and Checksums
- **Data Compression and Archives**
 - **gzip - compress or expand files**
 - **pigz - compress or expand files in parallel**
 - **bzip2 - block-sorting file compressor**
 - **tar - archiving utility**
- Data Transfer Tools
- Data Distribution Networks and Federations
- Questions & Answers (30 min)

Data Compression

Data compression is the process of encoding information using fewer bits than used in its original representation, reducing the resources required to store or transmit data. Different compression techniques are classified either as *lossless* or *lossy*.



Lossless compression techniques reduce bits by identifying and eliminating statistically redundant information. This allows the original data to be reconstructed from the compressed data with no information loss. Examples include Lempel-Ziv (LZ).

Lossy compression techniques reduce bits by removing unnecessary or less important information. This allows lossy methods to achieve higher compression ratios. Examples include discrete cosine transform (DCT).

gzip - compress or expand files

gzip is a file format and command-line tool for file compression and decompression. It utilizes a combination of [Lempel-Ziv \(LZ77\)](#) and [Huffman coding](#). Whenever possible, each file is replaced by one with the extension ***.gz**, while keeping the same ownership modes, access and modification times.

Decompress a file

```
$ gzip -d cifar-10-python.tar.gz      $ gunzip cifar-10-python.tar.gz
```

Compress a file

```
$ gzip cifar-10-python.tar
```

Decompress multiple files and keep the originals

```
$ gzip -dk cifar-10-matlab.tar.gz cifar-10-binary.tar.gz
```

Compress files with different speeds (or levels). The default is 6.

```
$ gzip -1 cifar-10-matlab.tar      $ gzip -9 cifar-10-binary.tar  
$ gzip --fast cifar-10-matlab.tar  $ gzip --best cifar-10-binary.tar
```

pigz - compress or expand files in parallel



pigz is parallel implementation of gzip for modern multi-processor, multi-core machines. Note, however, decompression can't be parallelized. As a result, pigz uses a single thread for decompression, but will create three other threads for reading, writing, and check calculation, which can speed up decompression in some cases.

Decompress a file

```
$ pigz -d cifar-10-python.tar.gz $ unpigz cifar-10-python.tar.gz
```

Compress a file

```
$ pigz cifar-10-python.tar
```

Decompress multiple files and keep the originals

```
$ pigz -dk cifar-10-matlab.tar.gz cifar-10-binary.tar.gz
```

Compress a file with different speeds (or levels) using a specific number of threads

```
$ pigz -1 -p2 cifar-10-matlab.tar $ pigz -9 -p4 cifar-10-binary.tar
```

bzip2 - block-sorting file compressor

bzip2

[bzip2](#) is another file format and command-line tool for file compression and decompression. It utilizes the [Burrows-Wheeler block sorting text compression algorithm](#) and [Huffman coding](#), which generally achieves better compression ratios than more conventional LZ77/LZ78-based compressors like gzip for most types of files.

Decompress a file

```
$ bzip2 -d cifar-10-batches-py.tar.bz2    $ bunzip2 cifar-10-batches-py.tar.bz2
```

Compress a file

```
$ bzip2 cifar-10-batches-py.tar
```

Decompress a file and keep the original

```
$ bzip2 -dk cifar-10-batches-py.tar.bz2
```

Check the integrity of the compressed file

```
$ bzip2 -tv cifar-10-batches-py.tar.bz2
```

gzip vs. bzip2

gzip is generally faster than bzip2, but the latter almost always results in smaller files. The degree of compression will depend on the nature of the data, with text being much more compressible than images

```
$ ls -l cifar-10-python.tar*          # 1.09x (gzip) 1.25x (bzip2)
-rw-r--r-- 1 sinkovit use300 186224640 Jun  4 2009 cifar-10-python.tar
-rw-r--r-- 1 sinkovit use300 147090541 Jun  4 2009 cifar-10-python.tar.bz2
-rw-r--r-- 1 sinkovit use300 170498091 Jun  4 2009 cifar-10-python.tar.gz

$ ls -l uniprot-proteome.fasta * # 1.82x (gzip) 2.08x (bzip2)
-rw-r--r-- 1 sinkovit use300 27415290 Mar 11 17:13 uniprot-proteome.fasta
-rw-r--r-- 1 sinkovit use300 13196498 Mar 11 17:13 uniprot-proteome.fasta.bz2
-rw-r--r-- 1 sinkovit use300 15065622 Mar 11 17:13 uniprot-proteome.fasta.gz

$ ls -l moby_dick.txt*                # 2.46x (gzip) 3.23x (bzip2)
-rw-r--r-- 1 sinkovit use300 1234609 Feb 10 10:50 moby_dick.txt
-rw-r--r-- 1 sinkovit use300  382785 Feb 10 10:50 moby_dick.txt.bz2
-rw-r--r-- 1 sinkovit use300   501265 Feb 10 10:50 moby_dick.txt.gz
```

tar - archiving utility

tar is a command-line tool for bundling a collection of many files into a single [archive file](#), which helps organize and make large datasets more manageable for other tasks like data compression, storage, and transmission. It also supports a number of different compression algorithms. *.tar files are commonly referred to as a *tarball*.

List the contents of an archive

```
$ tar -tf cifar-10-python.tar.gz $ tar --list --file cifar-10-python.tar.gz
```

Extract all of the files from an archive

```
$ tar -xf cifar-10-python.tar.gz $ tar --extract --file cifar-10-python.tar.gz
```

Create a new archive file

```
$ tar -cf cifar-10-batches-py.tar cifar-10-batches-py/
```

Create a new compressed archive file with gzip

```
$ tar -czf cifar-10-batches-py.tar.gz cifar-10-batches-py/
```

Create a new compressed archive file with bzip2

```
$ tar -cjf cifar-10-batches-py.tar.bz2 cifar-10-batches-py/
```

tar - avoid "tar bombs"

The **right** way to create an archive is to put your files into a directory and then apply tar to the directory. When extracting files, the directory structure will be reproduced.

The **wrong** way to create an archive is to put all the files on the command line. Upon extraction, the files will all be written to the current directory, potentially leaving you with a big mess to clean up.

```
$ ls MYFILES/  
file1.txt  file2.txt  file3.txt
```

```
$ tar -cf MYFILES.tar MYFILES/  
$ rm -rf MYFILES  
$ tar -xf MYFILES.tar  
$ ls  
MYFILES  MYFILES.tar
```



```
$ ls  
file1.txt  file2.txt  file3.txt
```

```
$ tar -cf MYFILES.tar file*.txt  
$ rm -f file*
```

```
$ ls  
MYFILES.tar  
$ tar -xf MYFILES.tar  
$ ls  
file1.txt  file2.txt  file3.txt  MYFILES.tar
```



Table of Contents

- Downloading Data from the Internet
- Data Integrity and Checksums
- Data Compression and Archives
- **Data Transfer Tools**
 - **scp - secure file copy**
 - **sftp - secure file transfer**
 - **rsync - remote (and local) file-copying tool**
 - **globus - GridFTP-based data transfer**
 - **rclone - syncs your files to cloud storage**
- Data Distribution Networks and Federations
- Questions & Answers (30 min)

scp - secure file copy

scp is a command-line tool to securely transfer (a copy of) files between a local computer system and remote system or two remote systems (or hosts). It relies on the [Secure Shell \(SSH\) protocol](#) and is included by default in almost every Unix-like operating system.

Upload a file from your local system to a remote host and rename the file

```
$ scp cifar-10-python.tar.gz mkandes@oasis-dm-interactive.sdsc.edu:~/cifar-10.tar.gz
```

Download a file from a remote host to your local system

```
$ scp mkandes@oasis-dm-interactive.sdsc.edu:~/cifar-10.tar.gz ./
```

Upload a directory from your local system to a remote host

```
$ scp -r cifar-10-batches-py/ mkandes@login.expense.sdsc.edu:~/
```

Transfer a directory from one remote host to another

```
$ scp -r login.expense.sdsc.edu:~/cifar-10-batches-py/ bridges2.psc.edu:~/
```

Upload a directory from your local system to a remote host with *compression enabled*

```
$ scp -C -r cifar-10-batches-py/ login.expense.sdsc.edu:~/cifar-10/
```

sftp - secure file transfer

sftp is an interactive command-line (client) tool that utilizes the [SSH File Transfer Protocol \(SFTP\)](#) to provide secure file access, management, and transfers between a local system and a remote server.

Open a connection to the remote system

```
$ sftp mkandes@login.expense.sdsc.edu  
Connected to login.expense.sdsc.edu.  
sftp>
```

List the contents of the pwd directory on the remote system

```
sftp> ls
```

Change directories on the remote system

```
sftp> cd cifar-10-batches-py/
```

Download a file from the remote system

```
sftp> get data_batch_1
```

Upload a file to the remote system

```
sftp> put cifar-10.txt
```

Close the connection to the remote system

```
sftp> exit
```

rsync



rsync is a command-line utility for transferring and *synchronizing* files and directories between two computer storage systems over a network. By default, it determines which files differ between the two by checking the modification time and size of the files on both ends.

Synchronize the contents of a directory with another on the same system

```
$ rsync -r cifar-10-batches-py/ cifar-10-batches-py_recursive_copy
```

Synchronize the contents of a directory in 'archive mode', which preserves almost all file properties

```
$ rsync -a cifar-10-batches-py/ cifar-10-batches-py_archive_copy
```

Synchronize the contents of a local directory with one on a remote system

```
$ rsync -a cifar-10-batches-py/ mkandes@login.expense.sdsc.edu:~/cifar-10-batches-py
```

Synchronize the contents of a remote directory with a local version

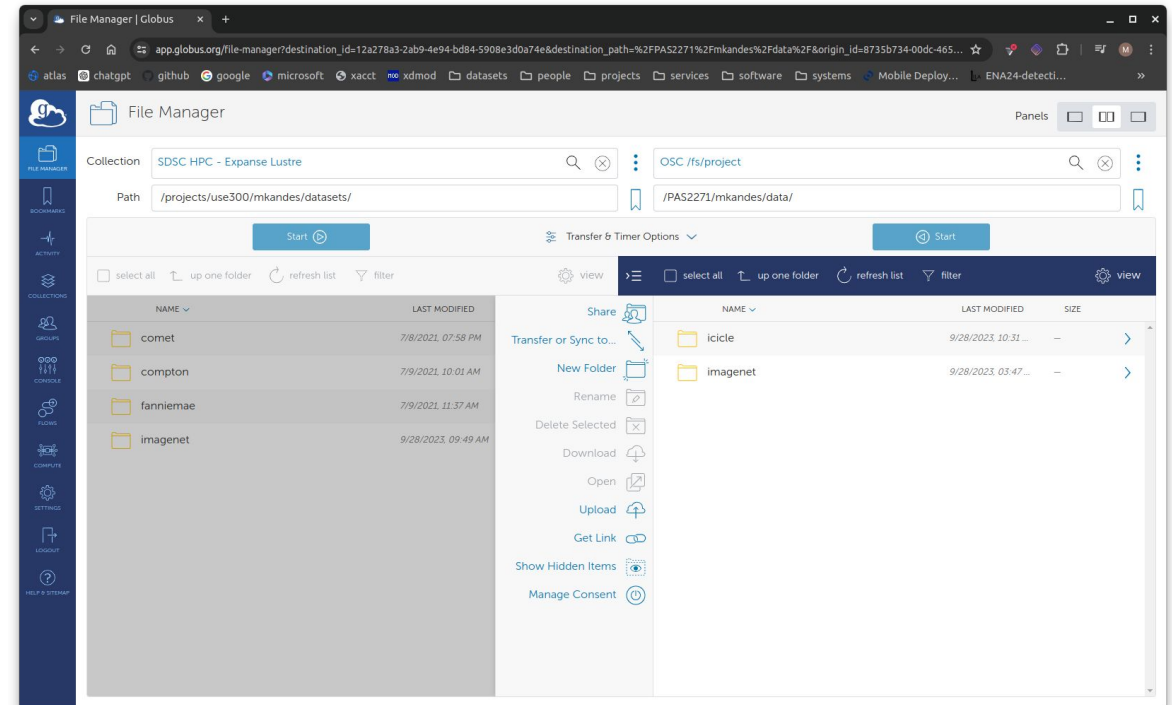
```
$ rsync -a mkandes@login.expense.sdsc.edu:~/cifar-10-batches-py ~/cifar-10-batches-py
```

Compress data before it is sent to the destination system

```
$ rsync -az cifar-10-batches-py/ mkandes@login.expense.sdsc.edu:~/cifar-10-batches-py
```

globus (GridFTP)

Globus is a non-profit organization run by the University of Chicago that develops and operates cyberinfrastructure products and services for research. Its most popular service is the Globus data transfer service, a GridFTP-based service, which provides a fast, secure, and reliable way to transfer large amounts of data — up to petabytes (PBs).



rclone - syncs your files to cloud storage



[rclone](#) is a command-line program to manage files on cloud storage like Amazon's S3 object storage service. Note, however, it supports more than 70 different storage backends with a number of advance capabilities. It also has POSIX filesystem support.

Copy files from source to dest, skipping identical files

```
$ rclone copy source:sourcepath dest:destpath
```

Make source and dest identical, modifying destination only

```
$ rclone sync source:sourcepath dest:destpath
```

Move files from source to dest

```
$ rclone move source:sourcepath dest:destpath
```

Mount the remote as file system on a mountpoint

```
$ rclone mount remote:path/to/files /path/to/local/mount
```

Serve a remote over a protocol

```
$ rclone serve http remote:
```

Table of Contents

- Downloading Data from the Internet
- Data Integrity and Checksums
- Data Compression and Archives
- Data Transfer Tools
- **Data Distribution Networks and Federations**
 - Open Science Data Federation (OSDF)
 - Open Storage Network (OSN)
 - National Science Data Fabric (NSDF)
- Questions & Answers (30 min)

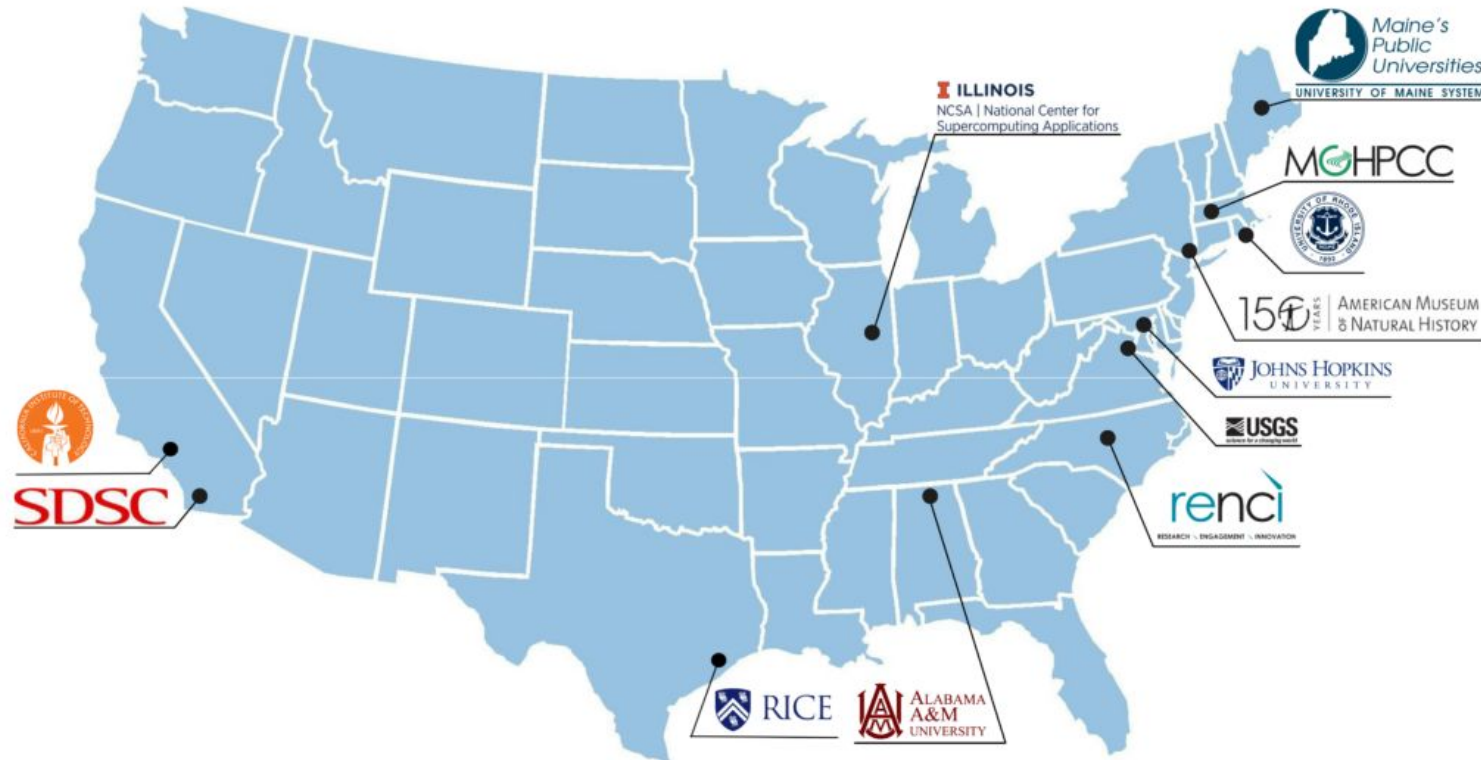
Open Science Data Federation (OSDF)

The [Open Science Data Federation \(OSDF\)](#) is an [Open Science Grid \(OSG\)](#) service designed to support the sharing of files staged in autonomous data “origins” to provide efficient access to those files from anywhere in the world via a global namespace and network of data storage caches.



Open Storage Network (OSN)

The [Open Storage Network \(OSN\)](#) is distributed network of storage systems (known as pods) that aim to help make data storage and transfer at scale simpler for scientific research.



National Science Data Fabric (NSDF)

The [National Science Data Fabric \(NSDF\)](#) is a platform agnostic testbed for integrated data delivery and access to shared storage, networking, and computing for scientific discovery.

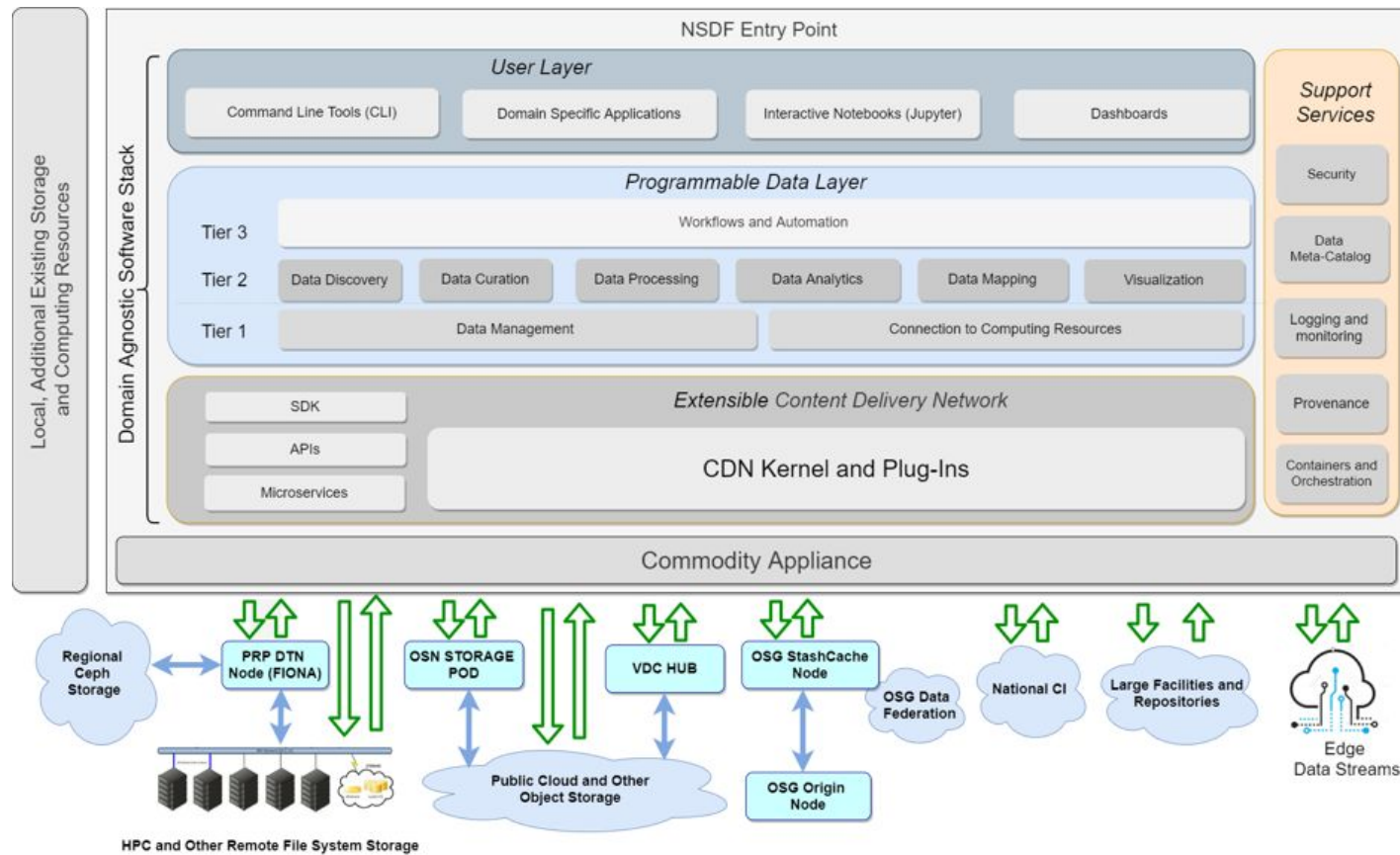


Table of Contents

- Downloading Data from the Internet
- Data Integrity and Checksums
- Data Compression and Archives
- Data Transfer Tools
- Data Distribution Networks and Federations
- **Questions & Answers (30 min)**

Additional Information

- [How To Use Wget to Download Files and Interact with REST APIs](#)
- [How to Download Files with cURL](#)
- [How to Use SCP Command to Securely Transfer Files](#)
- [How To Use SFTP to Securely Transfer Files with a Remote Server](#)
- [How To Use Rsync to Sync Local and Remote Directories](#)
- [Globus Guides](#)



Questions & Answers

